

## ЛАБОРАТОРНА РОБОТА №2. РОБОТА З ГРАФОВИМИ СТРУКТУРАМИ

**Мета та основне завдання роботи:** ознайомитися з бібліотеками для обробки графів **Spark GraphX** та **GraphFrames**, вивчити основні алгоритми центральності та пошуку шляхів. Набути практичних навичок аналізу зв'язків у великих масивах даних, використовуючи набори даних авіаперевезень **OpenFlights**. Навчитися виконувати з'єднання даних (**Join**) для підготовки атрибутів вершин та ребер графа.

### ОСНОВНІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Графова аналітика є потужним інструментом для розуміння зв'язків між об'єктами. У той час як стандартні методи аналізу фокусуються на агрегації та статистиці окремих записів, графові алгоритми розглядають структуру таких зв'язків.

Загальна програмна архітектура **GraphFrames** в **Apache Spark** базується на використанні високорівневих абстракцій **DataFrames** замість низькорівневих **RDD** (**Resilient Distributed Datasets**), що використовуються в старішому компоненті **GraphX**.

Ключові аспекти архітектури **GraphFrames**:

1. На відміну від **GraphX**, де графи представляються через **RDD** вершин та ребер, у **GraphFrames** граф (об'єкт **GraphFrame**) будується на основі двох **DataFrames**:

- **Vertices** (Вершини). **DataFrame**, який повинен містити колонку з назвою **id** для унікальної ідентифікації кожної вершини.
- **Edges** (Ребра). **DataFrame**, який повинен мати колонки **src** (джерело) та **dst** (призначення), значення яких посилаються на **id** у таблиці вершин. Крім обов'язкових колонок, ці **DataFrames** можуть містити будь-яку кількість додаткових колонок-атрибутів.

2. Інтеграція зі **Spark SQL** та оптимізація Оскільки **GraphFrames** побудовано на основі **DataFrames**, ця бібліотека тісно інтегрована з рушієм **Spark SQL**. Це забезпечує суттєві архітектурні переваги:

- **Catalyst Optimizer**. Запити до графа проходять через оптимізатор запитів **Catalyst**, який будує ефективний план виконання, чого не міг робити **GraphX** з **RDD**.

- Project Tungsten. Використання Tungsten дозволяє GraphFrames ефективніше управляти пам'яттю (використовуючи direct memory поза купою JVM) та генерувати оптимізований байт-код, що підвищує продуктивність у порівнянні з об'єктно-орієнтованим підходом GraphX.

3. Мовна підтримка Архітектура GraphFrames дозволяє використовувати бібліотеку не тільки в Scala (як GraphX), а й у Java та Python. Це стало можливим саме завдяки використанню DataFrame API, який є уніфікованим для різних мов.

### Створення графу

Імпорт даних в Apache Spark. Спочатку імпортуємо необхідні пакети зі Spark та пакету GraphFrames:

```
from graphframes import *  
from pyspark import SparkContext
```

Далі можна використати наступний код для створення GraphFrame на основі вмісту CSV-файлів:

```
v = spark.read.csv("data/social-nodes.csv", header=True)  
e = spark.read.csv("data/social-relationships.csv", header=True) g = GraphFrame(v, e)
```

### Алгоритми центральності

Алгоритми центральності використовуються для розуміння ролі окремих вузлів у графі та їхнього впливу на мережу. Вони дозволяють ідентифікувати найважливіші вузли, зрозуміти динаміку груп, швидкість поширення інформації та стійкість мережі [3].

**Ступенева центральність (Degree Centrality).** Ступенева центральність –це найпростіший алгоритм, який ми розглянемо в цій книзі. Він підраховує кількість вхідних і вихідних зв'язків вузла та використовується для визначення популярних вузлів у графі. Алгоритм ступеневої центральності був запропонований Лінтоном К. Фріменом у його статті 1979 року “Centrality in Social Networks: Conceptual Clarification”. Приклад візуалізації ступеневої центральності наведено на рисунку 2.1

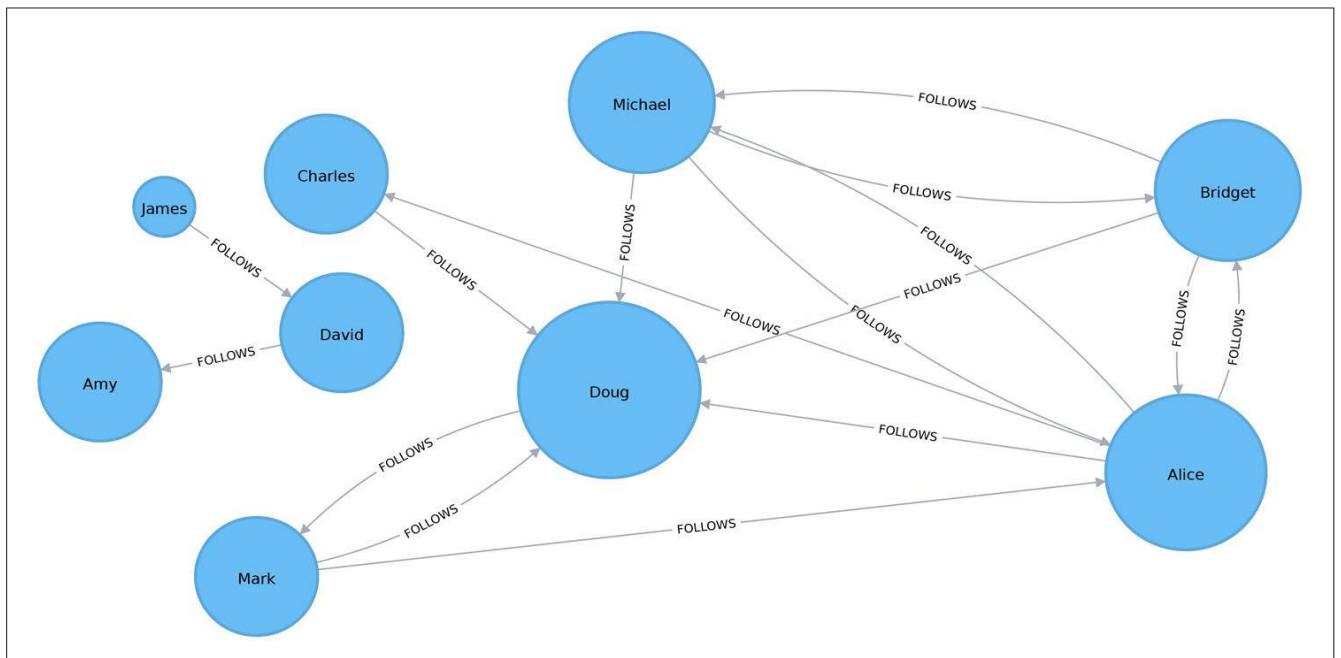


Рисунок 2.1 - Візуалізація ступеневої центральності

**Ступінь вузла** – це кількість прямих зв'язків з іншими вузлами. Розрізняють вхідний ступінь (in-degree) та вихідний ступінь (out-degree). Використовується для визначення "популярності" вузлів. Наприклад, у соціальних мережах користувач із високим вхідним ступенем (багато підписників) є впливовим, навіть якщо сам підписаний на малу кількість людей.

Середній ступінь мережі обчислюється як загальна кількість зв'язків, поділена на загальну кількість вузлів; він може бути значно зміщений у бік вузлів із високим ступенем. Розподіл ступеня показує ймовірність того, що випадково обраний вузол матиме певну кількість зв'язків.

Використовуйте центральність за ступенем, якщо вам потрібно оцінити вплив, аналізуючи кількість вхідних і вихідних зв'язків, або визначити "популярність" окремих вузлів. Вона добре працює, коли вас цікавить безпосередня зв'язність або ймовірність взаємодії у короткостроковій перспективі. Проте центральність за ступенем також застосовується для глобального аналізу, наприклад, для оцінки мінімального, максимального, середнього ступеня та стандартного відхилення у всій мережі.

Приклад реалізації центральності за ступенем в Apache Spark [3]

Виконаємо алгоритм центральності за ступенем за допомогою наступного коду:

```
total_degree = g.degrees
in_degree = g.inDegrees
out_degree = g.outDegrees

(
    total_degree
    .join(in_degree, "id", how="left")
    .join(out_degree, "id", how="left")
    .fillna(0)
    .sort("inDegree", ascending=False)
    .show()
)
```

Спочатку ми обчислюємо загальний ступінь, вхідний і вихідний ступені. Потім об'єднуємо ці DataFrame, використовуючи left join, щоб зберегти всі вузли, навіть якщо у них немає вхідних або вихідних зв'язків. Якщо вузли не мають зв'язків, ми встановлюємо їх значення на 0 за допомогою функції fillna.

**Closeness Centrality.** Цей алгоритм визначає вузли, які здатні найефективніше поширювати інформацію через мережу. Міра центральності вузла визначається як обернена величина до суми відстаней найкоротших шляхів від цього вузла до всіх інших вузлів. Використовуйте центральність близькості, коли потрібно визначити, які вузли поширюють інформацію найшвидше. Використання зважених зв'язків може бути особливо корисним при оцінці швидкості взаємодії в комунікаційних та поведінкових аналізах

**Betweenness Centrality.** Цей алгоритм виявляє вузли, які виконують роль «мостів» між різними частинами графа. Вузол має високу посередницьку центральність, якщо він лежить на великій кількості найкоротших шляхів між іншими парами вузлів. Застосовується для виявлення вузьких місць у логістиці, критичних точок у комп'ютерних мережах або впливових посередників у соціальних групах.

**PageRank** – найвідоміший алгоритм центральності, спочатку розроблений Google для ранжування веб-сторінок. Він вимірює транзитивний вплив: важливість вузла залежить не лише від кількості зв'язків, а й від важливості (рангу) тих вузлів, які на нього посилаються. Алгоритм ітеративно розподіляє вагу (ранг) вузла між його сусідами. Вводиться коефіцієнт демпфування (damping factor, зазвичай 0.85), який

моделює ймовірність того, що користувач продовжить перехід за посиланнями, а не перейде на випадкову сторінку. Застосовується для рекомендаційних системи, виявлення шахрайства, аналіз впливовості аеропортів у транспортній мережі.

Алгоритм PageRank з компоненти GraphFrames підтримує дві реалізації:

- Перша реалізація запускає PageRank для фіксованої кількості ітерацій. Це можна виконати, встановивши параметр `maxIter`.
- Друга реалізація запускає PageRank до конвергенції. Це можна виконати, встановивши параметр `tol`.

Розглянемо приклад підходу з фіксованою кількістю ітерацій:

```
results = g.pageRank(resetProbability=0.15, maxIter=20)
results.vertices.sort("pagerank", ascending=False).show()
```

В Apache Spark коефіцієнт демпінгу більш інтуїтивно називається ймовірністю скидання, з оберненим значенням. Іншими словами, ймовірність скидання = 0.15 в цьому прикладі є еквівалентною коефіцієнту демпінгу: 0.85 в Neo4j

### Пошук шляхів

Алгоритм пошуку в ширину (Breadth-First Search, BFS) BFS є одним із фундаментальних алгоритмів обходу графів і часто слугує основою для інших, більш складних алгоритмів аналізу мереж. Основний принцип роботи BFS полягає в тому, що він починає обхід з вибраного кореневого вузла і досліджує всіх його безпосередніх сусідів (вузли на відстані 1 кроку), перш ніж переходити до сусідів цих сусідів (вузли на відстані 2 кроків) і так далі.

Ця стратегія «віялоподібного» розширення робить BFS ідеальним інструментом для дослідження околиць вузла та пошуку шляхів у графах, де вага ребер не враховується або є однаковою для всіх зв'язків[3].

**Застосування BFS у незважених графах.** У контексті аналізу надвеликих масивів даних (Big Data), зокрема транспортних мереж чи соціальних графів, BFS використовується для знаходження найкоротших шляхів у незважених графах. У такому випадку «довжина» шляху вимірюється не фізичною відстанню (наприклад, кілометрами) чи вартістю, а кількістю ребер (зв'язків), які необхідно пройти, щоб

дістатися від початкового вузла до цільового. Цю метрику часто називають кількістю «хопів» (hops) або пересадок.

*Приклад:* При аналізі авіаперельотів BFS дозволяє знайти маршрут між двома аеропортами з мінімальною кількістю пересадок, ігноруючи фактичну відстань перельоту.

### **Ключові характеристики BFS.**

1. Пошук найкоротшого шляху (Shortest Path): у незваженому графі перший знайдений шлях від початку до цільового вузла за допомогою BFS гарантовано буде найкоротшим за кількістю кроків.

2. Аналіз досяжності: алгоритм дозволяє визначити, чи існує взагалі шлях між двома вузлами, що корисно для аналізу зв'язності компонентів графа.

3. Обмеження глибини: у реалізаціях для Big Data (наприклад, у Spark GraphFrames) BFS часто дозволяє задавати максимальну глибину пошуку (maxDepth), щоб уникнути надмірних обчислень у дуже великих графах.

Реалізація у Spark При використанні Spark GraphFrames, метод bfs дозволяє знаходити шляхи, що відповідають певному шаблону (motif), використовуючи підхід пошуку в ширину. Це дозволяє явно вказати початковий та кінцевий набори вузлів і знайти перший (найкоротший за кількістю ребер) шлях між ними. У GraphX для пошуку зважених найкоротших шляхів (наприклад, за алгоритмом Дейкстри) часто використовується API Pregel або ітеративний MapReduce (aggregateMessages), оскільки вбудований ShortestPaths працює лише з незваженими графами (кількість хопів).

Вибір алгоритму залежить від бізнес-задачі. Якщо мета - мінімізувати кількість пересадок або знайти "знайомих через одне рукостискання", використовується **BFS** (незважений граф). Якщо мета - мінімізувати час у дорозі або вартість пального, використовуються алгоритми **зваженого найкоротшого шляху** (наприклад, Дейкстри або A\*).

**Motif Finding** - це техніка пошуку підграфів або структурних шаблонів усередині великого графа. На відміну від простих запитів до бази даних, які фільтрують дані за значеннями атрибутів, пошук мотивів дозволяє фільтрувати дані

за їх структурою та зв'язками. У Spark GraphFrames для цього використовується декларативний підхід, де ви описуєте бажану структуру графа.

Синтаксис опису шаблонів. Для опису мотивів використовується текстовий формат, що нагадує візуальне зображення вершин і стрілок.

Приклад 1: Ланцюжок (A -> B -> C) — Рейс з однією пересадкою

Цей шаблон використовується для пошуку шляху від вершини A до вершини C через проміжну вершину B. У контексті авіаперельотів це відповідає рейсу з однією пересадкою. Синтаксис: "(a)-[e1]->(b); (b)-[e2]->(c)". Опис коду:

1. (a)-[e1]->(b): Існує ребро e1 від початкової вершини a до проміжної вершини b.
2. Символ “;” - це роздільник частин шаблону.
3. (b)-[e2]->(c): Від тієї ж проміжної вершини b існує ребро e2 до кінцевої вершини c.

```
# Пошук маршрутів з Польщі у Францію з одною пересадкою
motifs = g.find("(a)-[e1]->(b); (b)-[e2]->(c)")
result = motifs.filter("a.country = 'Poland' AND c.country = 'France'")
```

**Компоненти зв'язності (Connected Components)** - це алгоритми для виявлення зв'язних підграфів ("островів") у великому графі, де всі вузли пов'язані між собою шляхами, але не мають зв'язків з вузлами інших підграфів.

Типи зв'язності:

- Connected Components (Weakly): Знаходять групи, де вузли досяжні один з одного, ігноруючи напрямок зв'язків.
- Strongly Connected Components (SCC): Знаходять групи в орієнтованих графах, де кожен вузол досяжний з будь-якого іншого вузла тієї ж групи з урахуванням напрямку зв'язків (наприклад, взаємна підписка).

**Області застосування:**

- Кластеризація. Швидкий спосіб розділити дані на незалежні групи для подальшого аналізу.

- Соціальний аналіз. Виявлення ізольованих спільнот ("клік") або прогнозування соціальних кіл.
- Аналіз інфраструктури: Пошук розривів у мережах (наприклад, у комп'ютерних мережах або мережах постачання).

## **ПРАКТИЧНЕ ЗАВДАННЯ**

**Вхідні дані:** Для виконання роботи використовуються дані проекту [OpenFlights](#):

1. airports-extended.dat – інформація про аеропорти (ID, назва, країна, координати: широта/довгота).
2. airlines.dat – інформація про авіакомпанії.
3. routes.dat – інформація про маршрути (аеропорт вильоту, аеропорт прильоту, код авіакомпанії, кількість зупинок).

### **Завдання 1. Підготовка даних та розрахунок дистанцій.**

Завантажте дані у DataFrame/RDD. Використовуючи операцію Join, додайте географічні координати до таблиці маршрутів.

- Розрахуйте відстань для кожного рейсу (використовуючи формулу гаверсінуса).
- Знайти авіакомпанію з найбільшою сумарною довжиною рейсів.
- Знайдіть авіакомпанію з найменшою сумарною довжиною рейсів.

### **Завдання 2. Пошук маршрутів (Motifs / BFS)**

- Використовуючи Spark GraphFrames, побудуйте граф, де вершини – це аеропорти, а ребра – рейси.
- Знайти всі можливі рейси між країнами згідно варіантів у Додатку 2 даного посібника.
- Умови: маршрут повинен мати не більше 2-х стиковок (тобто 3 сегменти польоту).

Рекомендація: Використати пошук мотивів (motifs) або BFS. Порівняйте результати та складність реалізації.

### **Завдання 3. Аналіз найкоротших та найдовших маршрутів**

Знайти найкоротший та найдовший маршрут між 2 заданими аеропортами (на ваш вибір, згідно варіантів у Додатку 2 посібника). За метрику довжини брати фізичну відстань (км). Врахувати маршрути з пересадками (не більше 3 пересадок).

### **Завдання 4. Визначення впливовості (PageRank)**

Запустіть алгоритм PageRank на графі авіасполучень.

Знайти 5 найбільш впливових аеропортів. Проаналізуйте, чи співпадають вони з найбільшими хабами світу. Спробуйте змінити параметр resetProbability (damping factor) і проаналізуйте зміни у результатах.

### **Завдання 5. Кластеризація та аналіз зв'язності**

- Виділити кластери аеропортів. Використайте алгоритм пошуку зв'язних компонентів (Connected Components) або Label Propagation.
- Відфільтруйте та проаналізуйте кластери, що містять не менше 5 аеропортів.
- Знайти ізольовані підграфи: Знайдіть групи з кількістю вузлів не менше 2, які зв'язані між собою, але не мають зв'язків з іншими аеропортами (основною частиною мережі).

### **ВИМОГИ ДО ЗВІТУ**

Звіт до лабораторної роботи виконується у форматі Microsoft Word/PDF і повинен містити:

1. Титульний аркуш.
2. Відомості про виконавця.
3. Опис задачі та вхідних даних (з посиланням на джерело OpenFlights).
4. Опис реалізації:
  - Код завантаження та очищення даних.
  - Пояснення використаних алгоритмів (чому обрано саме ці методи).
  - Екранні форми виконання кожного завдання.
5. Аналіз результатів:
  - Яка авіакомпанія має найбільший кілометраж?

- Які аеропорти є найбільш впливовими за PageRank і чому?
- Опис знайдених кластерів та ізольованих підграфів.

## 6. Висновки.

Додатково до звіту надається посилання на файл ноутбука (.ipynb) або архів з кодом.

## СПИСОК КОНТРОЛЬНИХ ПИТАНЬ

1. У чому різниця між алгоритмами **Degree Centrality** та **PageRank**? Який з них краще визначає якість зв'язків, а не їх кількість?
2. Що таке **Betweenness Centrality** і як цей показник допомагає виявити "вузькі місця" в мережі авіасполучень?
3. Як впливає коефіцієнт демпфування (damping factor) на розрахунок PageRank? Що таке "rank sink"?
4. Які існують підходи до пошуку найкоротшого шляху у зваженому графі в Spark GraphX/GraphFrames?
5. Що таке **Connected Components** і як цей алгоритм використовується для кластеризації?

## ДОДАТОК Б

### Варіанти завдань до лабораторної роботи 2

Рекомендований список варіантів завдань надаються для ознайомлення та може бути змінений.

Варіант розраховується за формулою: остання цифра порядкового номеру у навчальній групі.

| Варіант | Знайди маршрути між країнами |
|---------|------------------------------|
| 1       | Україна – Польща             |
| 2       | Польща – Бельгія             |
| 3       | Німеччина – Велика Британія  |
| 4       | Велика Британія – Франція.   |
| 5       | Велика Британія – Іспанія.   |
| 6       | Німеччина – Іспанія          |
| 7       | Іспанія – Португалія.        |
| 8       | Швеція – Норвегія.           |
| 9       | Італія – Франція             |
| 0       | Франція – Іспанія            |